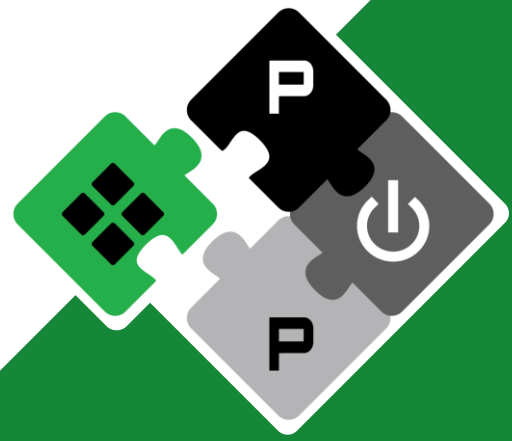
ETH zürich ALMA MATER STUDIORUM UNIVERSITÀ DI BOLOGNA

CachePool: Many-core cluster of customizable, lightweight scalar-vector PEs for irregular L2 data-plane workloads

Integrated Systems Laboratory (ETH Zürich)

Zexin Fu, Diyou Shen zexifu, dishen@iis.ee.ethz.ch

Alessandro Vanelli-Coralli avanelli@iis.ee.ethz.ch
Luca Benini lbenini@iis.ee.ethz.ch



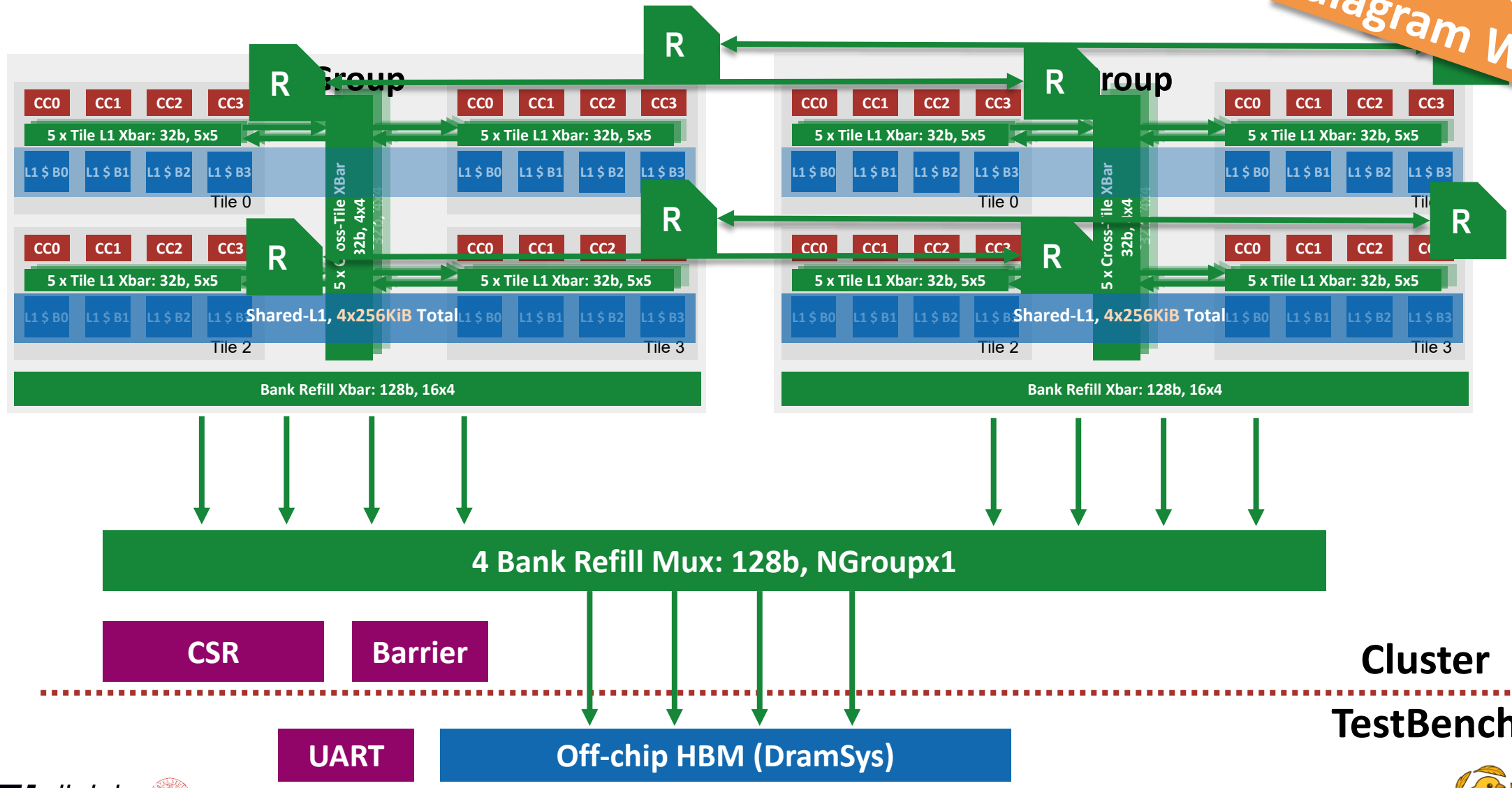
@pulp_platform

pulp-platform.org

youtube.com/pulp_platform

PULP Platform
Open Source Hardware, the way it should be!

Hardware Development (Cluster)



Hardware Development (Cluster)

- **Group NoC Wrapper / Cluser**
  - Multiple bug fixes
  - Add 4x4 configuration (16 Groups => 256 Snitch + Spatz4 CC)
    - Simulation speed is quite slow now
    - Testing undergoing
- **iCache**
  - Add L2 iCache at each Group to reduce instruction traffics
- **Refill NoC**
  - Similar NoC design as L1
  - Wider (128b), less parallel NoCs
  - HBM or DRAM (easily switchable)?

Hardware Development (L1 Cache)

- Folded cache + forwarding buffer Rebasing
  - Rebase the dev branch to the main branch (xbar-based multi-tile stable version)
  - Checked SpyGlass lint
    - Bit width mismatch
    - CombLoop
      - read_valid → write-enable (tcdm_wrapper) → bank grant (tile fold arbiter) → read_valid
      - read and write shared one case(status), so the read feeds the write-enable that drives the grant that gate the read
      - Cut: extract the write-issue (bank_req_write/addr/data/mask) out of the case (write has priority over read in the case of SRAM access), driven only by write_has_data → write-enable no longer depends on the read → feedback broken.

Hardware Development (L1 Cache)

- Cache controller debug fixing
  - **MSHR drain-count overflow:** an off-by-one counter width silently dropped miss responses under heavy load → cores hang.
    - The non-blocking cache packs many outstanding misses ("MSHR sub-entries") into a pending cache line.
    - A counter tracks how many sub-entries to drain when the refill returns — but it was one bit too narrow.
    - Under MSHR-full pressure the count 7 + 1 wrapped to 0 instead of 8 → the drain was gated off → all pending responses for that line were silently dropped → the waiting cores never get their data and hang.

Hardware Development (L1 Cache)

- Cache controller debug fixing
  - **Store read-after-write visibility bug:** the cache acks a write before the data is actually stored into cache → a second core can read the stale value.
    - A store is acknowledged to the core at request acceptance, not at SRAM commit (at that time, the write req is in the wr_req_fifo)
    - For a few cycles after the ack, the new value is still draining through the write pipeline, acked but not yet visible by other requests.
    - The new read request will not check the writes inside the wr_req_fifo, instead it will directly read data from cache SRAM, causing RAW harzard.
    - Failure pattern: two cores increment a lock-protected shared counter (load → store → unlock).
      - Core A's store is acked, it releases the lock
      - The lock variable is in a near cache bank, while the counter variable is in a far cache bank
      - Core B reads the counter from SRAM while A's store write is in the wr_req_fifo → reads the old value → both write the same value → the counter never advances → unbounded over-production → hang.

GVSoC Modeling Progress

- Started building a GVSoC performance model for the Insitu cache as the first step toward a full CachePool GVSoC model.
- The current model already captures the main cache-side behaviors:
  - Set-associative tag/data arrays with configurable ways, sets, and line size
  - Basic cache-line states: VALID, READ_PEND, WRITE_PEND
  - MSHR-based pending-request merging
  - Refill, eviction, and write handling
  - Write-back and write-through modes
- A standalone GVSoC cache testbench has been set up and can run initial smoke tests and synthetic access-pattern tests.
- Hash-based way selection and folded-cache support are still under development and will be parameterized for easy enable/disable.

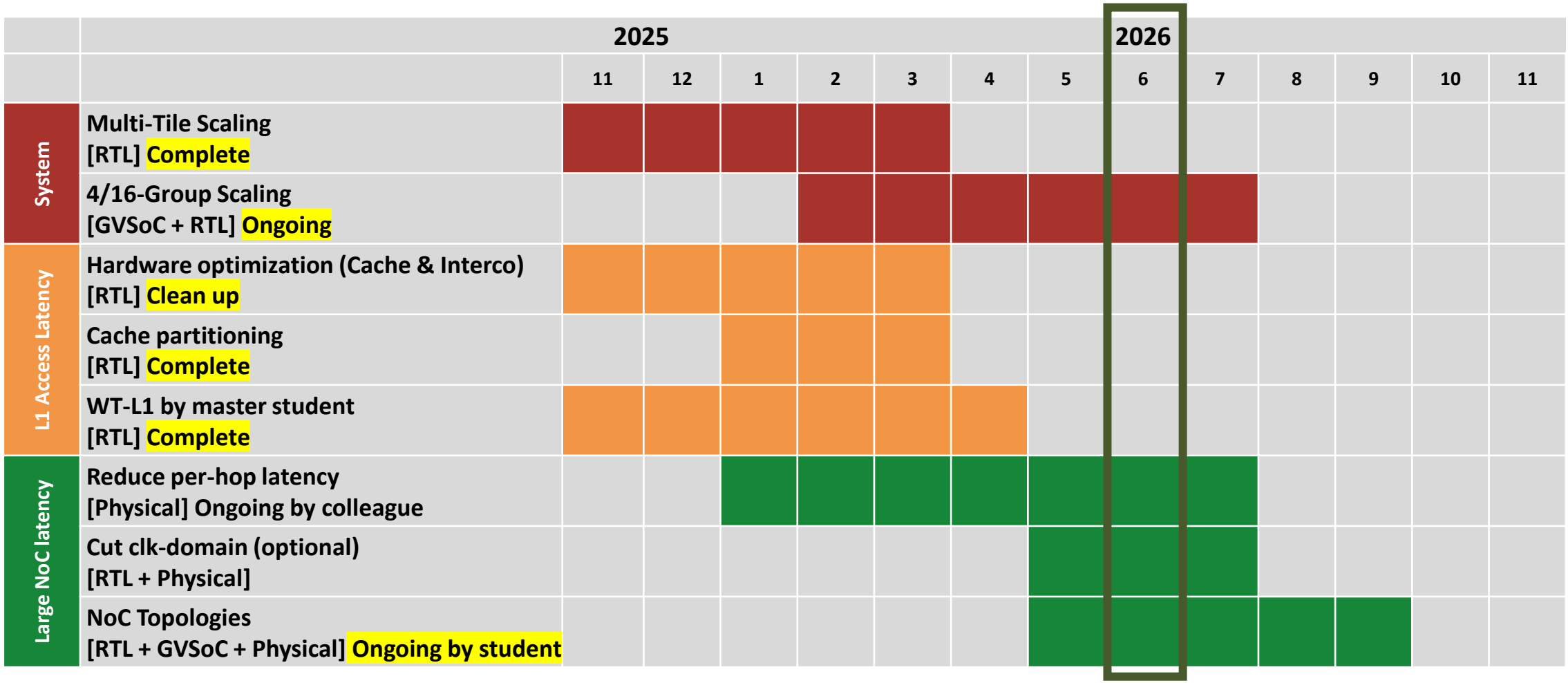
RTL-GVSoC Calibration Plan

- A standalone RTL testbench for the Insitu cache controller has now been set up.
- Both RTL and GVSoC sides now have similar standalone validation environments.
- The L2 memory is currently modeled as a fixed-latency SRAM to keep the memory-side behavior deterministic.
- Next step: run the same memory-access traces on both RTL and GVSoC and directly compare performance behavior.
- Key calibration metrics:
  - Hit latency
  - Miss/refill latency
  - Throughput
  - Outstanding-miss behavior
- After standalone calibration, the next target is a more complete tile-level model including Snitch + Spatz CC, tile interconnect, Insitu cache banks, and cache-to-memory interconnects.

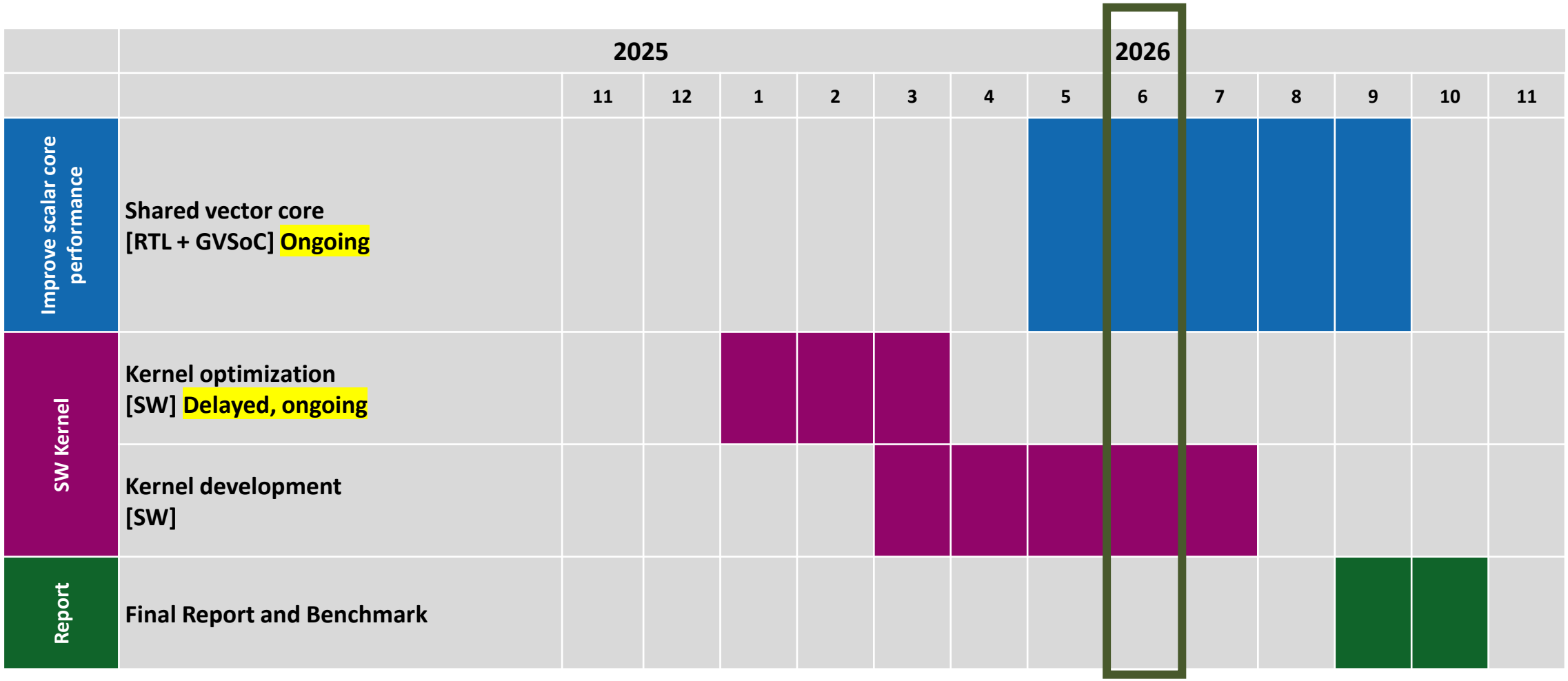
Next Step

- **InSitu-Cache Paper**
  - Both Zexin and Diyou will focus on the InSitu cache paper in May (ICCD). The progress on other side might be delayed
- **Diyou Side**
  - 7nm Flow (ongoing)
  - Debug and improve on multi-group
    Add refill NoC
- **Zexin Side**
  - Continue the insitu cache GVSoC model
  - Develop the RLC kernel for multi-user use cases

Timeline


2025
2026
11
12
1
2
3
4
5
6
7
8
9
10
11
System
Multi-Tile Scaling
[RTL] Complete
4/16-Group Scaling
[GVSoC + RTL] Ongoing
L1 Access Latency
Hardware optimization (Cache & Interco)
[RTL] Clean up
Cache partitioning
[RTL] Complete
WT-L1 by master student
[RTL] Complete
Large NoC latency
Reduce per-hop latency
[Physical] Ongoing by colleague
Cut clk-domain (optional)
[RTL + Physical]
NoC Topologies
[RTL + GVSoC + Physical] Ongoing by student

Timeline


2025
2026
11
12
1
2
3
4
5
6
7
8
9
10
11
Improve scalar core performance
Shared vector core
[RTL + GVSoC] Ongoing
SW Kernel
Kernel optimization
[SW] Delayed, ongoing
Kernel development
[SW]
Report
Final Report and Benchmark

L1 Access Latency

- **Problem: increased L1 access latency when scaling up**
- **Solutions**
  - **(Ongoing, RTL & Physical) Hardware optimization**
    Further optimize the interconnection and cache controller to reduce access latency on existing architecture
    => Retiming the current interconnection, remove unnecessary cuts in interco and cache
    => Target to reduce 1-2 cycles (~5 cyc) for hit-to-use latency
    Currently has reduced to 6 cycles:
    Core => Xbar => AMO => Ctrl (2) => AMO => Core
  - **(Complete, RTL) Cache partitioning**
    Partition the shared L1 to keep data localized to avoid remote access latency
  - **(Complete with some problems, RTL & Physical) WT-L1**
    Explore and evaluate a private WT L1 for each core
    => Performance degrade on single tile
    => Needs more work to support larger system

L1 Access Latency

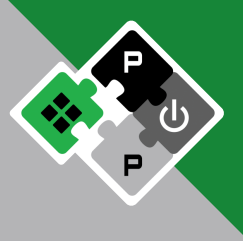
- **Problem: large NoC latency (~28 cyc based on previous exploration)**
- **Solutions**
  - **(Ongoing, Physical) Reduce hop latency**
    Explore the timing under 7nm technology to see the possibility of reducing per-hop latency to 1 cycle
    => Reduce the latency by 1/2
  - **(Planned, Physical) Cut clk-domain (optional)**
    If 1-cyc is not possible, explore to operate the NoC under a higher frequency than cores (e.g. 1.5GHz)
    => Effectively cut the latency by 1/3
  - **(Planned, GVSoC & Physical) NoC Topology**
    Explore different NoC topologies
    => Torus to reduce the diameter of NoC by 1/2, reduce longest latency by 1/2

PE-Related

- **Problem: Improve scalar core performance**
- **Solutions**
  - **(Planned, RTL & GVSoC) Shared vector core**
    Explore two Snitch sharing one Spatz configuration to improve scalar performance
    Software complexity needs to be considered, especially on the possible conflicting use of register file.
    Discussed Solution: critical section

RLC Workload

- **Problem: Current RLC model cannot reveal all properties of the kernel**
- **Solutions**
  - **(Ongoing, SW) Kernel optimization**
    Adapt the current model to better represent the real cases, e.g. do some operations on the data. Further cooperation with HQ side to develop a more complete and better model.
  - **(Planned, SW) Kernel development**
    Implement the missing use cases for multi-user



Thank you!